AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

Crash Course

(ONE SHOT Revision)

Unit-4

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Pdf Notes I AKTU PYQs I Imp. Questions

BCS101 / BCS201: PROGRAMMING FOR PROBLEM SOLVING SYLLABUS

Unit – 4

Functions: Introduction, Types of Functions, Functions with Array, Passing Parameters to Functions, Call by Value, Call by Reference, Recursive Functions.

Basic of Searching and Sorting Algorithms: Searching & Sorting Algorithms (Linear Search, Binary Search , Bubble Sort, Insertion and Selection Sort)

Gateway Classes

Empowering Learners, Trasforming Futures

AKTU

For Free Notes.....Download the Gateway Classes App

Helpline No-7819 0058 53, 7455 9612 84

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-4: Lecture-1

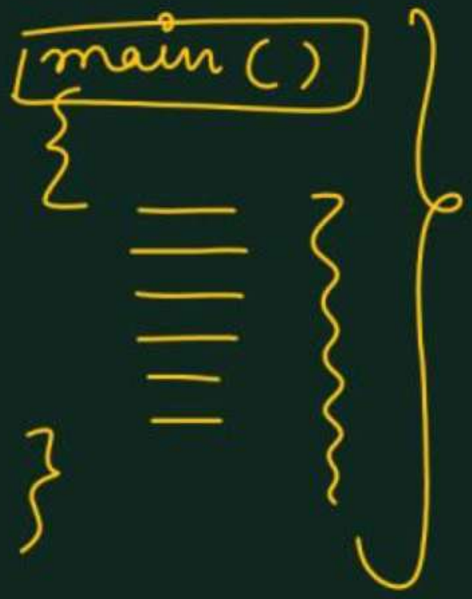
(PART-1 + PART 2)

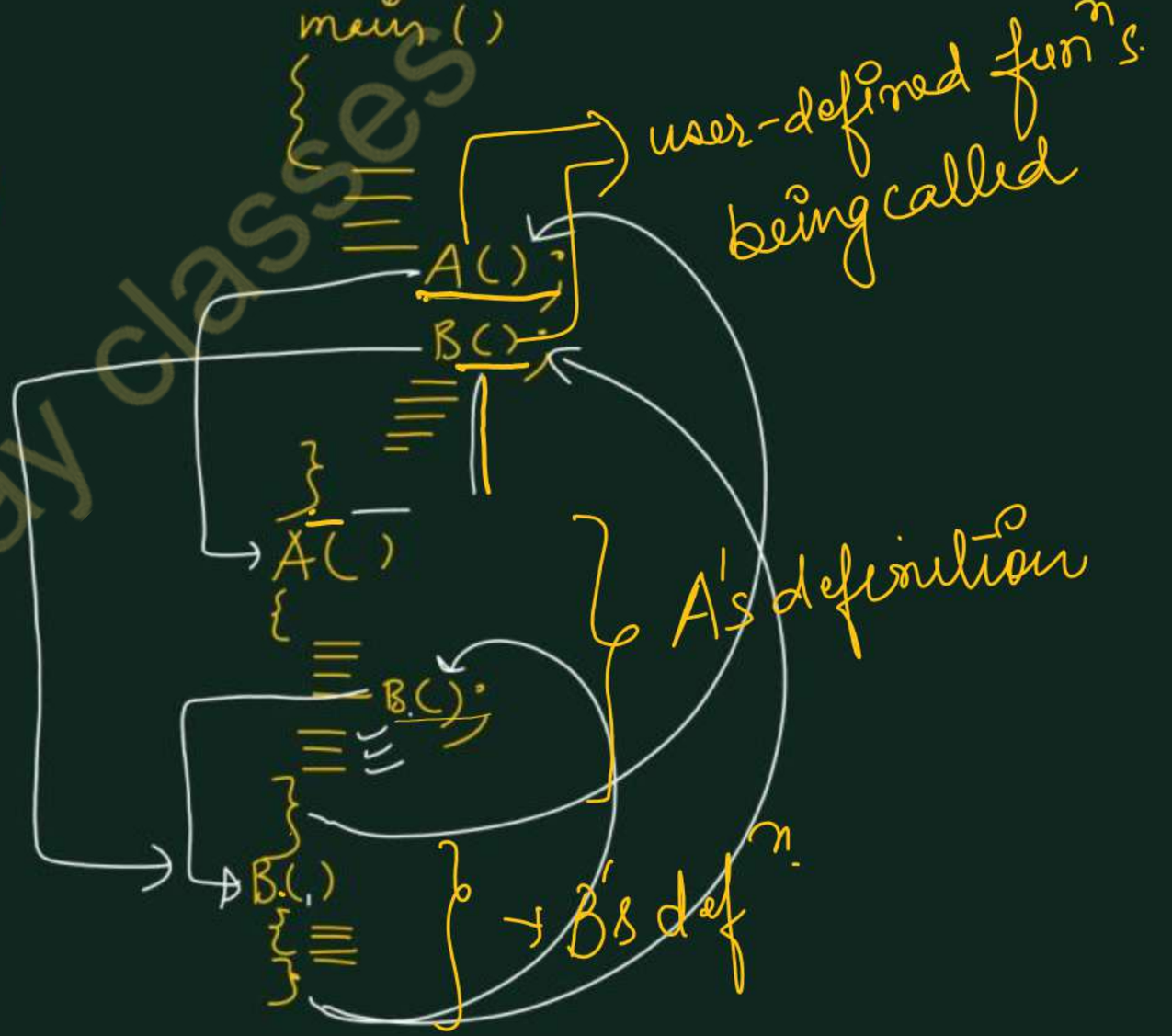
Topics to be Covered

- Function in C
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified





Functions in C (AKTU)

A function is a self-contained subprogram that is meant to do some specific, well-defined task. A program consists of one or more functions. If a program has only one function, then it must be the main function.

Advantages of using functions (AKTU)

a) To improve the readability of code.

b) Improves the reusability of the code, same function can be used in any program rather than writing the same code from scratch.

c) Debugging of the code would be easier if you use functions, as errors are easy to be traced.

d) Reduces the size of the code, duplicate set of statements are replaced by function calls.

C programs have two types of functions

1. Library functions (Built-in functions)
2. User-defined functions

Library Functions: (Built-in functions)

These functions are present in the C library and they are predefined. in different header files

For example:

- sqrt( ) is a mathematical library function which is used for finding out the square root of any number.
- The functions scanf( ) and printf( ) are input output library functions. Similarly, we have functions like strlen( ), strcmp( ) for string manipulation.

User Defined Functions:

- Users can create their own functions for performing any specific task of the program. These types of functions are called user-defined functions.
- To create and use these functions, we should know about these three things

1. Function definition
2. Function declaration
3. Function call

Default return type of a user-defined function in C is "int"

int

Sum(int, int);

Program to find the sum of two numbers using function

With arg with return

```
#include<stdio.h>
int sum(int, int);                          // Function Declaration
int main()// calling function
{
    int a, b, result;
    printf("Enter two numbers");
    scanf("%d%d",&a,&b);
    result= sum(a,b);                       // function call
    printf("The sum of two numbers is=%d",result);
    return 0;
}
```

a = 5

b = 10

result = 15

control Tranfer

```
int sum(int x, int y)                       // called function
{
    int z;// local variable
    z=x+y;
    return(z);
}
```

x and y are formal parameters

x = 5

y = 10

z = 15

Function definition

a and b are actual parameters / Actual Arguments

output

1) with argument with return value
2) without argument with return
3) without argument without return
4) with argument without return

With Argument without return

```
#include<stdio.h>
void sum(int,int);
int main()
{ int a,b;
 printf("Enter two numbers");
 scanf("%d%d",&a,&b);
    sum(a,b);
 return 0;
}
void sum(int x,int y)
{
 printf("The sum is=%d",(x+y));
}
```

Without Argument with return

```
#include<stdio.h>
int sum();
int main()
{
   int result;
   result = sum();
  printf("Sum is=%d",result);
 return 0;
}
int sum()
{ int a,b,c;
 printf("Enter two numbers");
  scanf("%d%d",&a,&b);
     c=a+b;
     return(c);
}
```

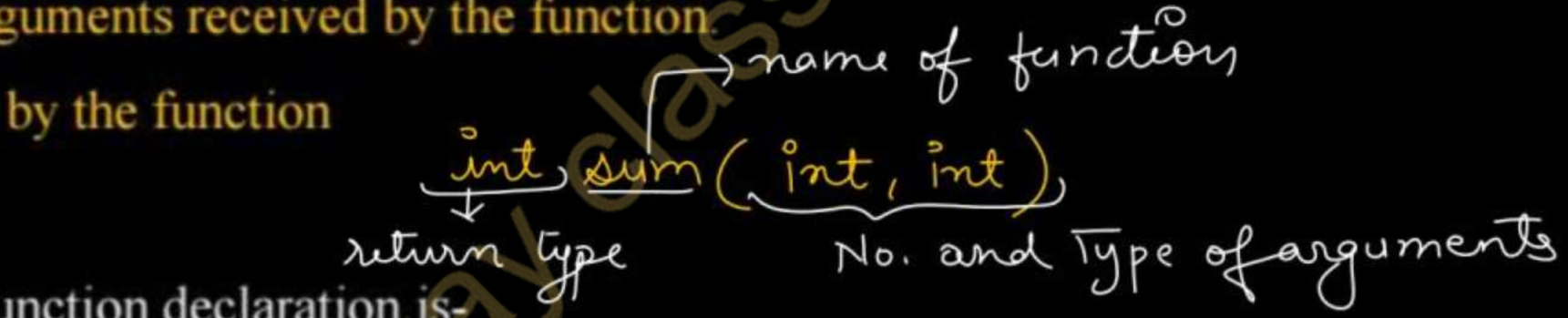
Without argument without Return

```
#include<stdio.h>
void sum();
int main()
{
     sum();
 return;
}
void sum()
{
 int a,b;
printf("Enter two numbers");
 scanf("%d%d",&a,&b);
printf("The sum is=%d",(a+b));
}
```

Function Declaration:

The function declaration is also known as the function prototype and it informs the compiler about the following three things.

1. Name of the function
2. Number and type of arguments received by the function
3. Type of value returned by the function



The general syntax of a function declaration is-

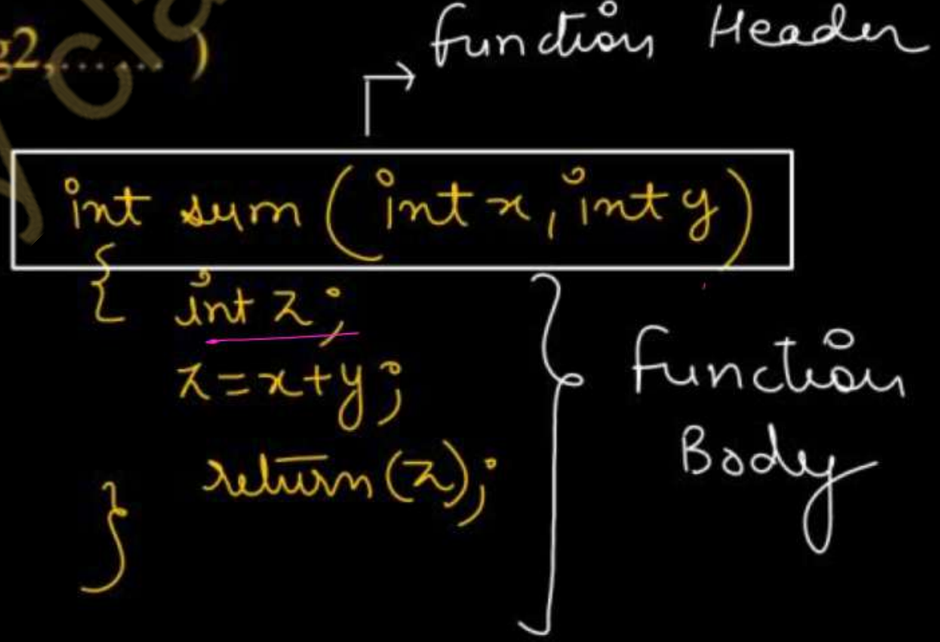
```
return_type func_name(type1 arg1, type2 arg2, .....);
```

The calling function needs information about the called function. If definition of the called function placed before the calling function, then declaration is not needed.

Function Definition:

- The function definition consists of the whole description and code of a function.
- It tells what the function is doing and what are its inputs and outputs.
- A function definition consists of two parts - a function header and a function body.
- The general syntax of a function definition is

```
return_type func_name ( type1 arg1, type2 arg2,..... )
{
    local variables declarations;
    statement;
    return(expression) ;
}
```

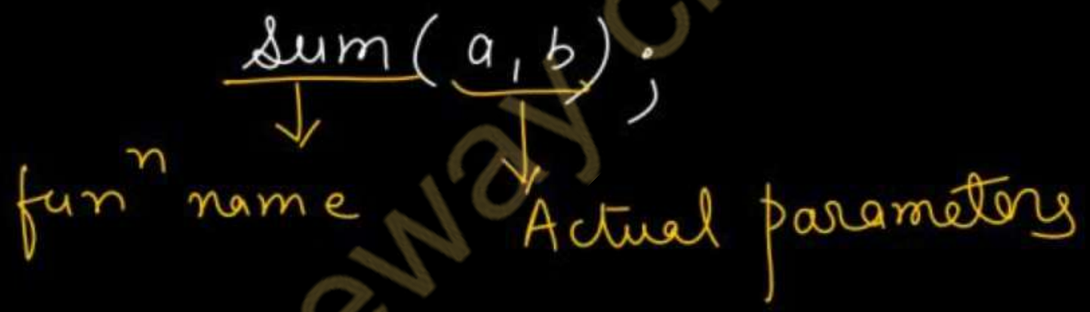


Function Call:

A function is called by simply writing its name followed by the argument list inside the parentheses

func_name(arg1, arg2, arg3 ...)

These arguments arg 1, arg2, ,...are called **actual arguments**.



return statement

→ return is a keyword.

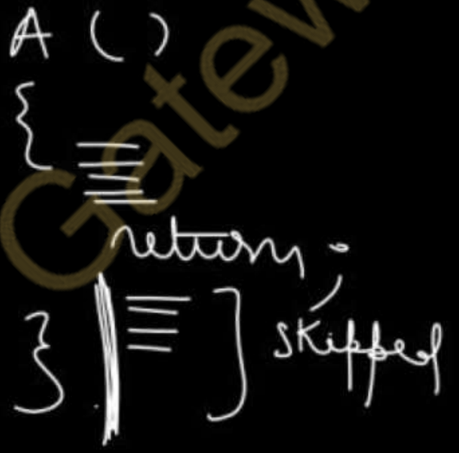
- return statement is used to return some value or simply pass the control to the calling function.
- It can be used in the following two ways.

1. return;

//used to terminate the function and pass the control to the calling function without returning a value.

2. return expression;

//used to return values from a function. The expression following return can be any constant, variable, function call etc.



return statement can return a single value at a time

Valid / Invalid return statements

1) return ; ✓

2) return (a); ✓

3) return a ; ✓

4) return 0; ✓

5) return 4.5; ✓

6) return (a+b); ✓

7) return (a,b); ✗

/*Program that finds maximum of two*/
using function

```
#include<stdio.h>
int max(int,int);
int main()
{
   int a, b, big;
   printf("Enter two numbers");
   scanf("%d%d",&a,&b);
      big = max(a,b);
   printf("The bigger number is=%d",big);
   return 0;
}
```

→ call

```
int max(int x, int y)
{
   if(x>y)
      return(x);
   else
      return(y);
}
```

A function definition can have more than one return statements but only one of them gets executed.

Program to compute the factorial of a number using a function.

```
#include <stdio.h>
int factorial(int);
int main()
{
  int n, fact;
  printf("Enter the number");
  scanf("%d", &n);
  if(n<0)
  {
    printf("factorial of negative no. is
                not defined");
  }
  else
  {
    fact = factorial(n);
    printf("The factorial is = %ld", fact);
  }
  return 0;
}
```

`fact = factorial(n);` → `factorial(n);`

```
int factorial(int n)
{
  int i, f=1;
  for(i=1; i<=n; i++)
  {
    f = f * i;
  }
  return(f);
}
```

5
1×2×3×4×5

Program to compute reverse of a number using a function.

```
#include<stdio.h>
int reverse(int);
int main()
{
  int n, result;
  printf("Enter the number");
  scanf("%d",&n);

  result=reverse(n);
  printf("The reverse no is=%d",
                          result);

  return 0;
}
```

```
int reverse(int n)
{
   int d, rev=0;
   while(n>0)
   {
     d=n%10;
     rev=rev*10+d;
      n=n/10;
   }
   return(rev);
}
```

DPP

1) WAP to check a number is armstrong or not using a user defined func^{n}.

2) WAP to check whether the given year is leap or not using fun^{n}.

/* Program to check for Armstrong no using function */

```
#include <stdio.h>
void Armstrong(int);
int main()
{
  int n;
  printf("Enter the number");
  scanf("%d",&n);
  Armstrong(n);
  return 0;
}
```

```
int Armstrong(int n)
{
  int x, d, sum=0;
  x = n;
  while(n>0)
  {
    d = n%10;
    sum = sum + d*d*d;
    n = n/10;
  }
  if(sum==x)
    printf("Armstrong");
  else
    printf("Not Armstrong");
}
```

n=0

Difference between actual and formal parameters (AKTU)

Basis	Actual Parameter	Formal Parameter
Definition	They are actual values passed to a function on which the function will perform operations	They are the variables in the function definition that would receive the values when the function is invoked (called)
Occurrence	It occurs when we ~~invoke~~ call a function	It occurs when we define a function
Provided by	Either by the programmer or by user	Only by programmer
Data types	Data types are not mentioned with the actual parameters sum(a,b)	Data types are mentioned along the formal parameters int sum(int x, int y)
Form	It can be a value or a variable	It is always a variable
Example	add (a, b);	int add (int x, int y) {//body}

AKTU PYQs

1. What are the different types of functions in C programming.(AKTU 2023-24)
2. What is a function? Why programmers use functions in code? While executing a function, how the values are passed between calling and called environment.(AKTU 2018-19)
3. Explain function prototype. Why it is required?(AKTU 2017-18)
4. What do you mean by function prototype?(AKTU 2021-22, 2016-17)
5. What do mean by user-defined function. Explain how to write user-defined fun^n's in C program using fun^n declaration, function call and function definition.

AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-4: Lecture-2

Topics to be Covered

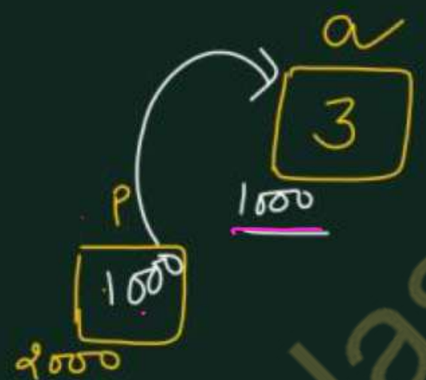
- Parameter passing methods
- Call by value and Call by reference
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

pointer (A variable that stores address of another variable)

int a = 3;



Declaring a pointer

```
int *p;
```

p is an integer pointer.

OR

p can hold address of an integer variable.

OR

p can point to an integer.

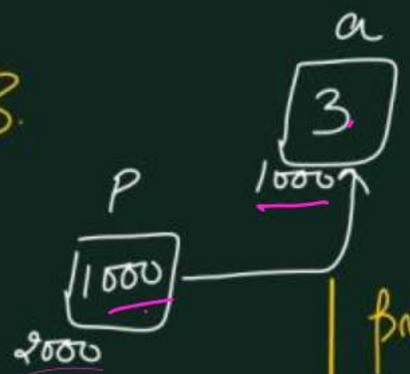
```
int *p, a = 3;
p = &a;
```

address of

```
char *p;
float *p;
```

```
int *p, a=3;
p=&a;
```

a, *p } ⇒ 3



Two operators

1) Address of operator (&)
2) Value at address operator (dereferencing operator) (*)

*p ⇒ Means value at address stored in p.

address in p ⇒ 1000

value at 1000 is 3 ✓

So *p ⇒ 3

```
printf("%d", a);   // 3
printf("%d", &a);  // 1000
printf("%d", p);   // 1000
printf("%d", *p);  // 3
printf("%d", *(&a));// 3
printf("%d", &p);  // 2000
```

Program to swap values of two variables using call by value parameter passing method

```
#include<stdio.h>

void swap(int, int);
int main()
{ int a,b;
  printf("Enter two numbers");
  scanf("%d%d",&a,&b);
  printf("The values before swapping
        are=%d and %d",a,b);
  swap(a,b);
  return 0;
}
```

a: 10

b: 20

swap(a,b); → call by value

values of a and b are passed in the function call

```
void swap(int x, int y)
{ int temp;
  temp=x;
  x=y;
  y=temp;
  printf("The values after swapping are %d and %d",
         x,y);
}
```

x: 10 → 20

y: 20 → 10

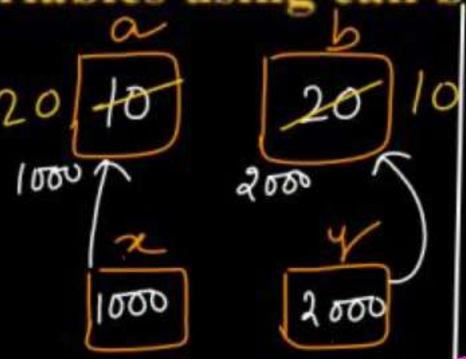
temp: 10

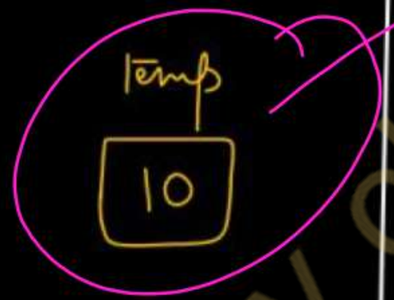
All the changes are done in x and y. As in case of call by value, all the changes are made to the copies of actual parameters but not on actual parameters, so the changes in formal parameters does not reflect back to the actual parameters in the calling fun".

Program to swap values of two variables using call by reference parameter passing method

```
#include<stdio.h>
void swap(int*, int*);
int main()
{
  int a, b;
  printf("Enter two numbers");
  scanf("%d%d", &a, &b);
  printf("The values before swapping are=%d and
         %d", a, b);
  swap(&a, &b);
  printf("The values after swapping are %d and %d",
         a, b);
  return 0;
}
```

Addresses are passed in function call.






```
void swap(int *x, int *y)
{
  int temp;
  temp = *x;
  *x = *y;
  *y = temp;
}
```

pointers are used as formal parameters

All the changes done in the function def^n are reflected back in the actual arguments as we work on same variables (not on their copies)

Imp **Difference between call by value and call by reference** (AKTU)

No.	Call by value	Call by reference
1	A copy of the value is passed into the function call.	An address of value is passed into the function call
2	Changes made inside the function definition is limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function definition validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.
3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location
4	No pointers are used.	Pointers are used.
5	It requires more memory	It requires less memory

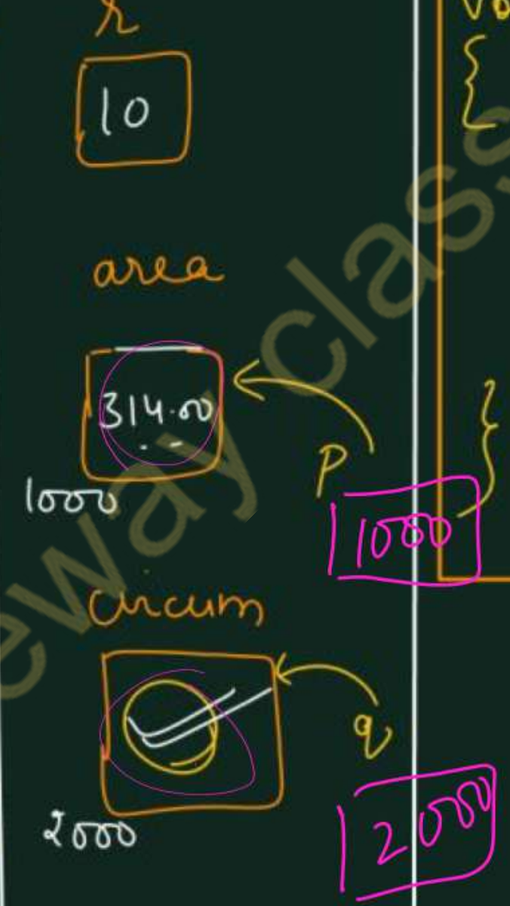
WAP to compute area and circumference of a circle using radius provided by the user with the help of call by reference parameter passing in function. (4K)

```
#include<stdio.h>
void circle(float, float*, float*);
int main()
{
  float r, area, circum;
  printf("Enter radius");
  scanf("%f",&r);
  circle(r,&area,&circum);
  printf("Area is=%f", area);
  printf("Circumference=%f", circum);
  return 0;
}
```

```
void circle(float r, float *p, float *q)
{
  *p = 3.14*r*r;
  *q = 2*3.14*r;
}
```

3.14 * 100.0 ⇒ 314.00

2 x 3.14 * 10.0



AKTU PYQs

1. Differentiate between call by value and call by reference parameter passing methods.

(AKTU 2022-23,2021-22)

1. What do you mean by call by value and call by reference? Write a program to read two integers X and Y and swap the contents of the two variables X and Y using pointers.

(AKTU 2021-22, 2018-19)

1. Differentiate between call by value and call by reference. Write a program in C that computes the area and circumference of a circle with radius taken as input using call by reference in functions.

(AKTU 2019-20)

AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-4: Lecture-3

V. Imp

Topics to be Covered

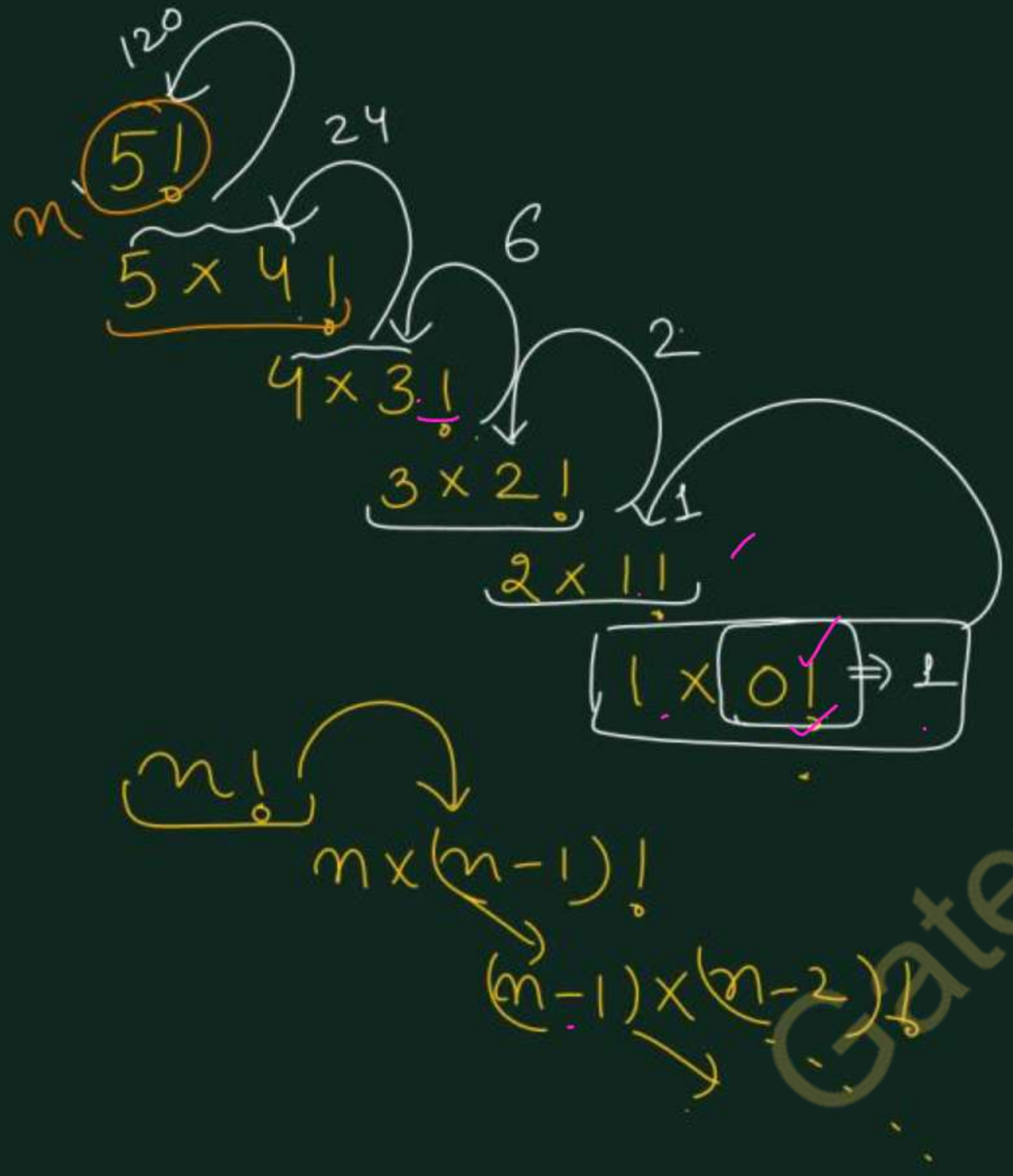
- Recursion in C
- Programs related to recursion
- AKTU PYQs

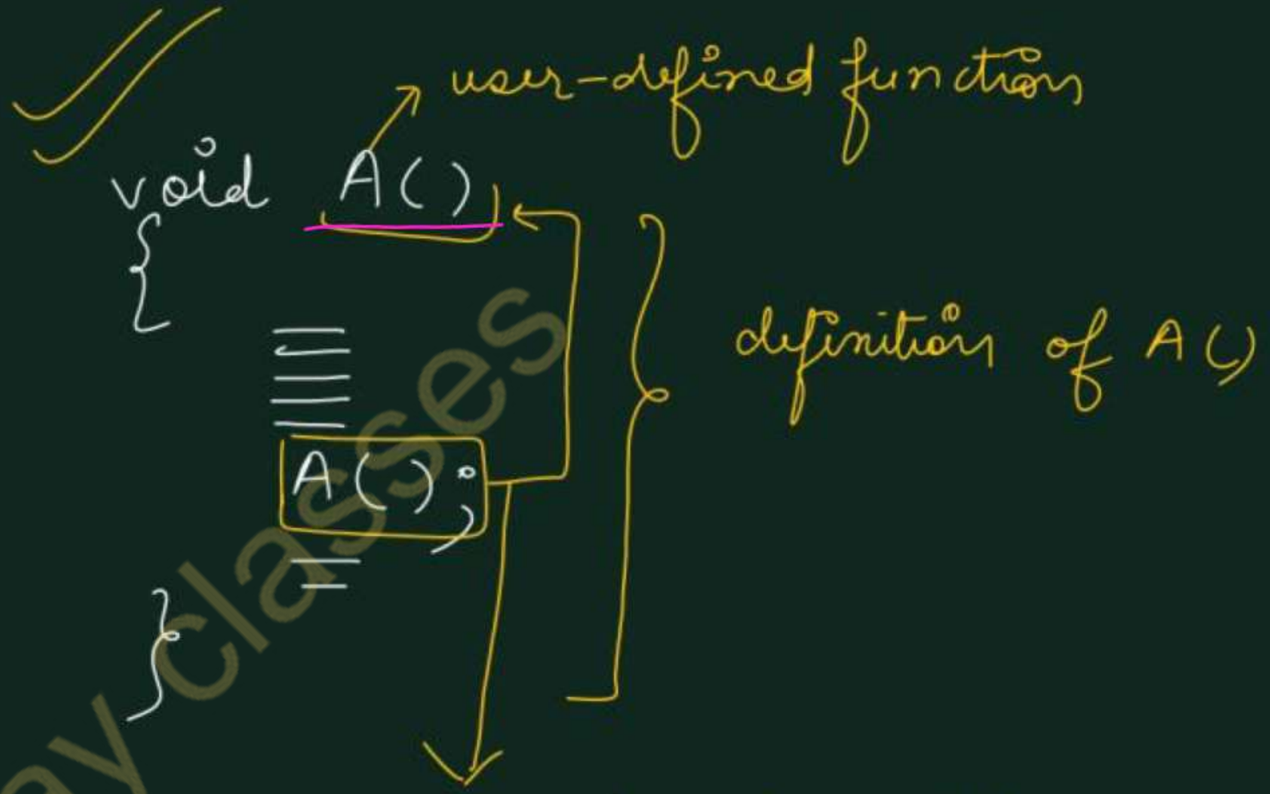
By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Recursion

- Recursion is the process of a function calling itself repeatedly till the given condition is satisfied.
- The function that calls itself (inside function body} again and again is known as recursive function and such kind of function calls are called recursive calls.
- In recursion the calling function and the called function are same.
- In C, recursion is used to solve complex problems by breaking them down into simpler sub-problems.
- We can solve large numbers of problems using recursion in C. For example, factorial of a number, generating Fibonacci series etc..





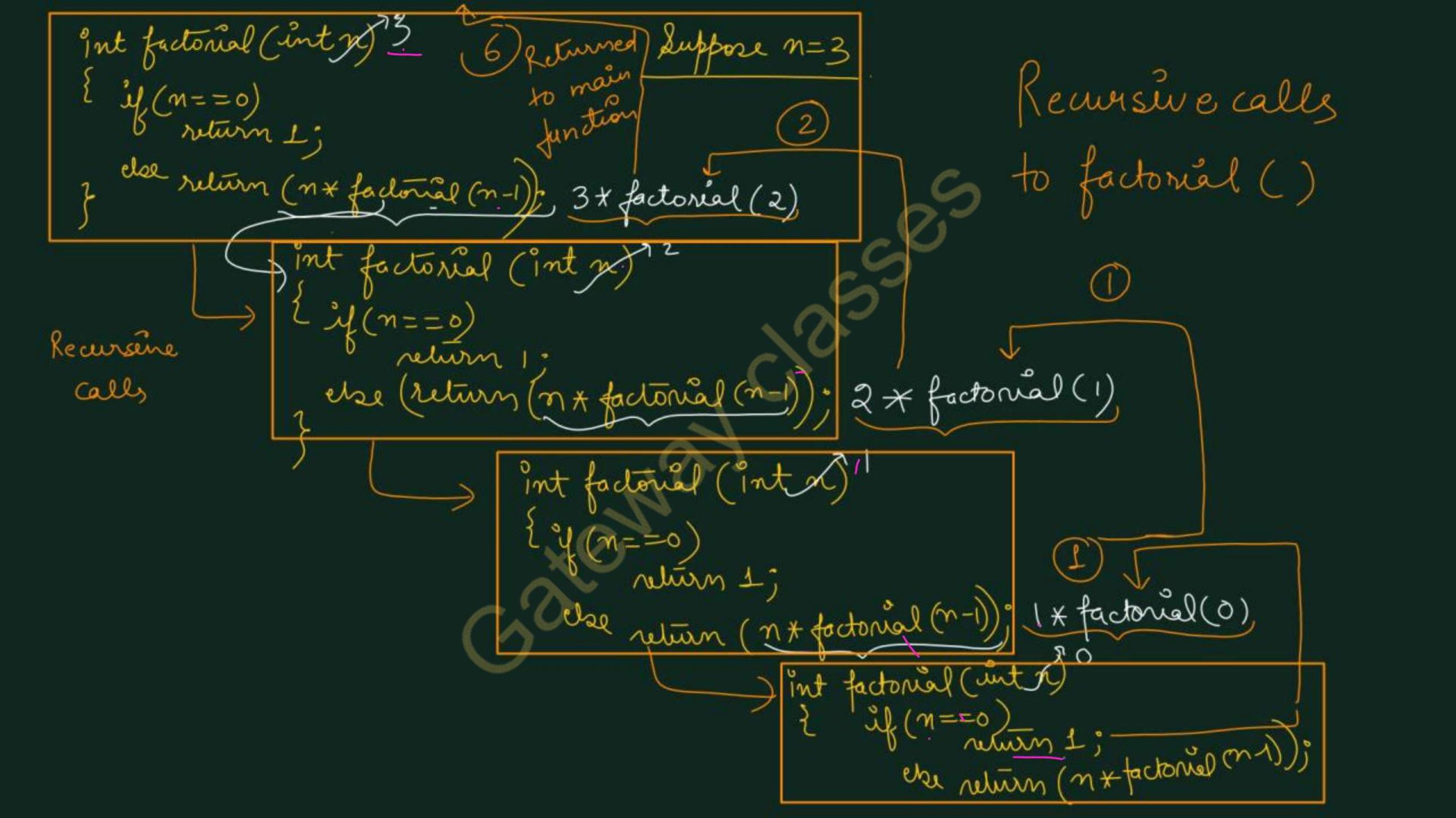
program in C to find factorial of a number using recursion(AKTU 2022-23,2020-21)

```
#include<stdio.h>
int factorial(int);
int main()
{
  int n, result;
  printf("Enter the number");
  scanf("%d",&n);
  result= factorial(n);
  printf("The factorial is=%d",result);
  return 0;
}
```

```
int factorial(int n)
{
  if(n==0)
    return(1);
  else
    return(n * factorial(n-1));
}
```

n-1

Base Condition


int factorial (int x)
{ if (n==0)
return 1;
else return (n * factorial (n-1));
}
Suppose n=3
3 * factorial (2)
2 * factorial (1)
1 * factorial (0)
Recursive calls to factorial ( )
Recursive calls
Returned to main function
6
1
2
3
Gateway classes

Program to print the Fibonacci series using recursive function.(AKTU 2021-22, 2020-21, 2017-18)

```
#include<stdio.h>
int fib(int);
int main()
{ int n, i;
  printf("Enter how many terms to print");
  scanf("%d",&n);
  for(i=0; i<=n-1; i++)
  {
    printf("%d\t", fib(i));
  }
  return 0;
}
```

0 1 1 2 3 5 ...

i

fib(i-2) fib(i-1)

0 1 1

```
int fib(int i)
{ if(i==0)
    return 0;
  if(i==1)
    return 1;
  return((fib(i-1)+fib(i-2)));
}
```

fib(1) + fib(0)

1 0

Program to find sum of N natural numbers using recursive function.(AKTU 2021-22)

```
#include<stdio.h>
int sum(int n);
int main()
{ int n;
  printf("Enter how many natural numbers to add");
  scanf("%d",&n);
  printf("Sum of the numbers is =%d", sum(n));
  return 0;
}
```

```
int sum(int n)
{ if(n==0)
    return 0;          } Base condition
  else return (n+sum(n-1));
}
```

Let n=4

return (4 + sum(3))

return (3 + sum(2))

return (2 + sum(1))

return (1 + sum(0))

0 is returned in main()

Program in C to find GCD of two numbers using recursion(AKTU 2022-23)

Greatest common divisor (HCF)

12 = 1, 2, 3, 4, 6, 12

30 = 1, 2, 3, 5, 6, 10, 15, 30

GCD / HCF = 6

Program in C to find GCD of two numbers using recursion(AKTU 2022-23)

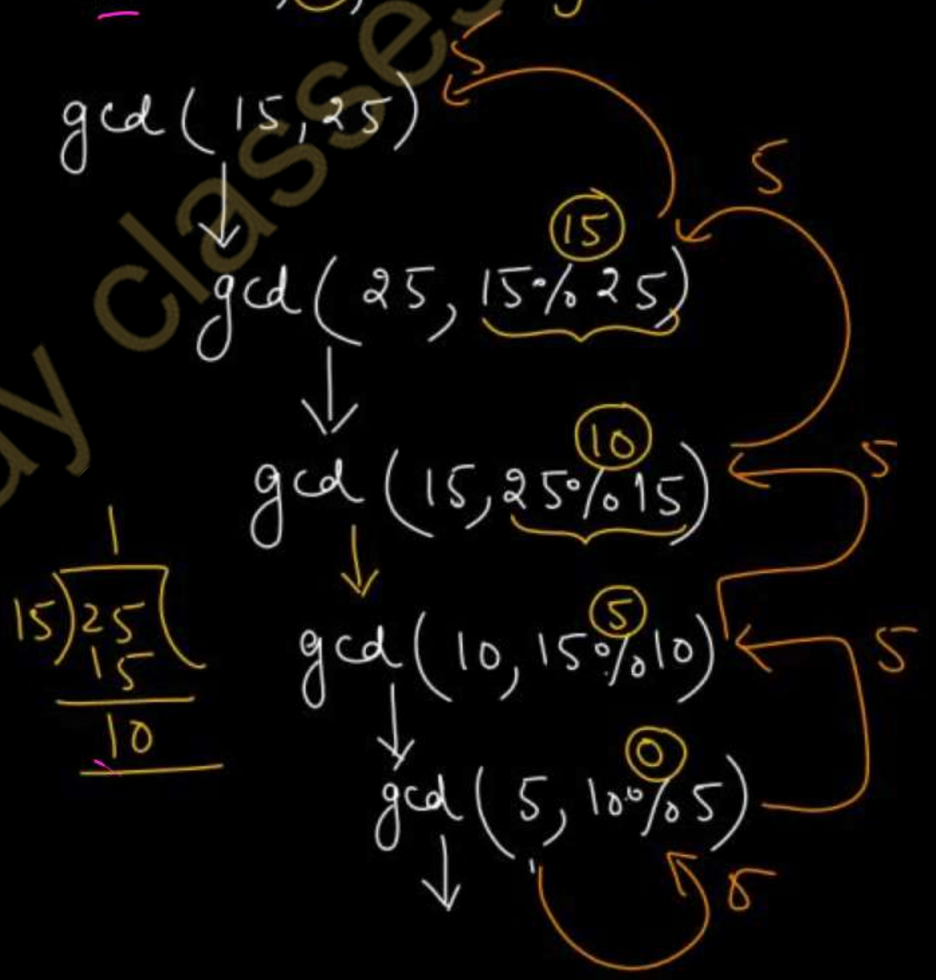
```c
#include<stdio.h>
int gcd(int,int);
int main()
{ int a,b;
 printf("Enter two numbers");
 scanf("%d%d",&a,&b);
printf("GCD is=%d",gcd(a,b));
 return 0;
}
int gcd(int a,int b)
{  if(b==0)
       return a;
   else
       return(gcd(b,a%b));
}
```

bare condition

lets assume a = 15 and b = 25

15 = 1, 3, 5, 15
25 = 1, 5, 25

} gcd(15,25) ⇒ 5

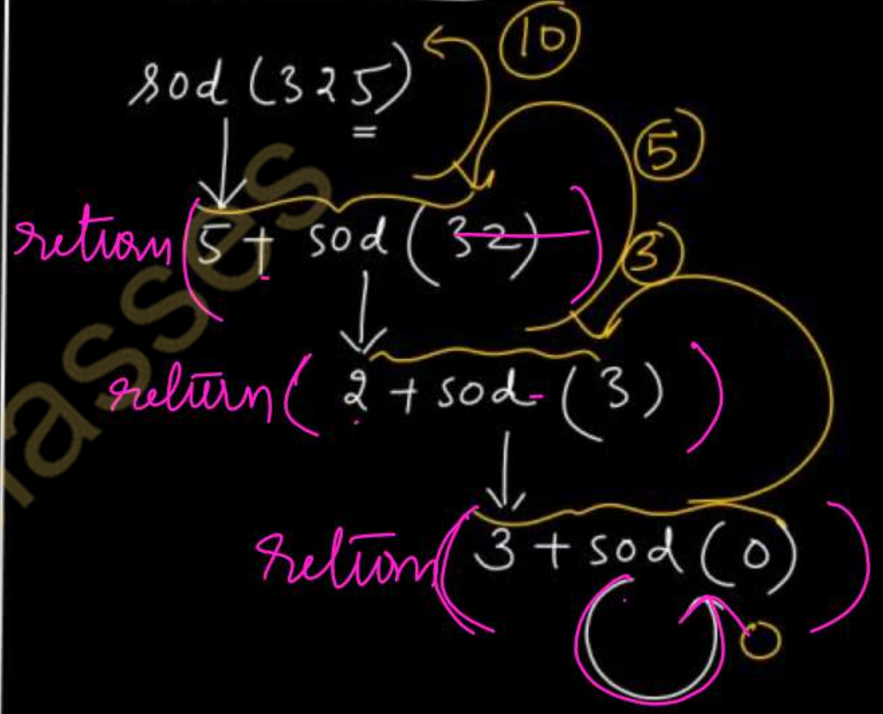


Program to find sum of digits of a number using recursion.

```
#include<stdio.h>
int sod(int);
int main()
{
  int n;
  printf("Enter the number");
  scanf("%d",&n);
  printf("The sum of digits is=%d",sod(n));
  return 0;
}
```

```
int sod(int n)
{
  if(n==0)
      return 0;
  else return(n%10+sod(n/10));
}
```

Let n=325



Advantages and Disadvantages of Recursion

Advantages:

- It makes code easier to write.
- Solves naturally recursive problems.
- Reduce time complexity.

Disadvantages:

- More use of Memory (It uses stack)
- Recursion can be slow: it requires the allocation of a new stack frame.
- Difficult to analyze code

Difference between Recursion and Iteration (AKTU)

Basis	Recursion	Iteration(Loops)
Basic	Calling a function itself within its own code.	Execute instructions repetitively until condition is false.
Termination	Termination condition is defined within the recursive function.	Termination condition is defined in the definition of the loop. for(i=0; i<=10; i++)
Code size	Code size is smaller	The code size is larger
Applied	It is always applied to functions.	It is applied to loops.
Speed	It is slower than iteration.	It is faster than recursion.
Usage	Used where no issue of time complexity, and code size requires being small.	It is used when we have to balance the time complexity against a large code size.
Complexity	It has high time complexity.	The time complexity is relatively lower.
Stack	It has to update and maintain the stack.	There is no utilization of stack.
Memory	It uses more memory.	It uses less memory as compared to recursion.

AKTU PYQs

1. Illustrate recursion. Write a program in C to find GCD of two numbers using recursion(AKTU 2022-23)
2. Explain recursion. Write a program in C to find factorial of a number using recursion(AKTU 2022-23,2020-21)
3. Define recursion. Write a program to print the Fibonacci series using recursive function.(AKTU 2021-22, 2020-21, 2017-18)
4. Write a program to find sum of N natural numbers using recursive function.(AKTU 2021-22)
5. Differentiate recursion and iteration.(AKTU 2018-19)

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-4: Lecture-4

Topics to be Covered

- Sorting
  - Bubble Sort
  - Insertion Sort
  - Selection Sort
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

AKTU PYQs

1. Define sorting. Explain the bubble sort technique and write the program to implement it. (AKTU 2023-24, 2018-19)
2. Write the program to implement selection sort. (AKTU 2023-24, 2021-22)
3. Write a program in C to implement bubble sort. (AKTU 2022-23)
4. Why is sorting process required?(AKTU 2022-23)
5. Explain the bubble sort technique with the help of example list {5,7,2,1,3,6}. (AKTU 2023-24)
6. Explain the selection sort technique with the help of an example. (AKTU 2020-21)

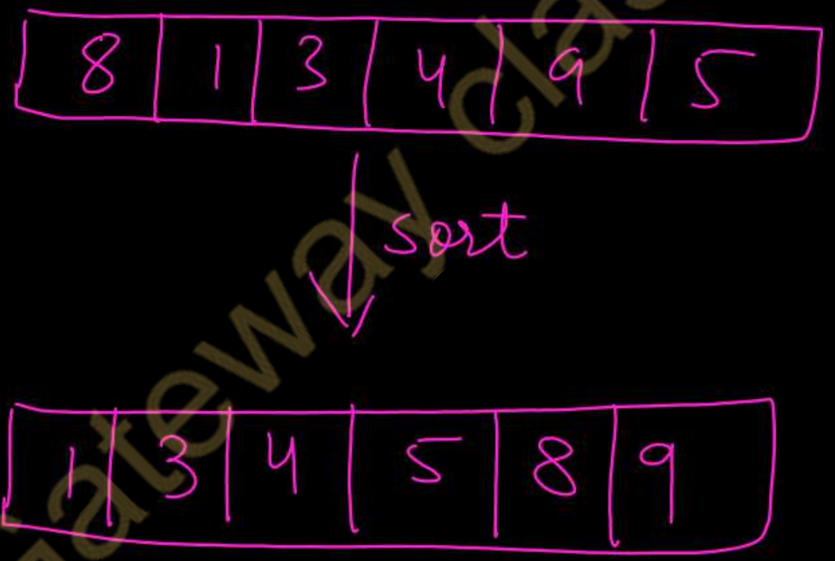
Sorting

Sorting is a technique to arrange the elements of a list in ascending or descending order.
There three basic algorithms to sort an array.

1) Bubble Sort

2) Selection Sort

3) Insertion Sort



Bubble Sort

$A = \{40, 20, 50, 60, 30, 10\}$

$n = 6$, no. of passes = 5

n elements list → Max (n−1) passes to sort the list

i=0 Pass-1 | i=1 pass-2 | i=2 Pass-3 | i=3 Pass-4 | i=4 Pass-5

outer for loop

inner loop

	j=0	j=1	j=2	j=3	j=4	j=0	j=1	j=2	j=3	j=0	j=1	j=2	j=0	j=1	j=0	Sorted array
0	40	20	20	20	20	20	20	20	20	20	20	20	20	20	20	10
1	20	40	40	40	40	40	40	40	40	40	40	30	30	30	10	20
2	50	50	50	50	50	50	50	50	30	30	30	40	10	10	30	30
3	60	60	60	60	30	30	30	30	50	10	10	10	40	40	40	40
4	30	30	30	30	60	10	10	10	10	50	50	50	50	50	50	50
5	10	10	10	10	10	60	60	60	60	60	60	60	60	60	60	60
	Ex.			Ex.				Ex			Ex.	Ex.		Ex.	Ex	

/* Program to sort an array using bubble sort*/

```
#include<stdio.h>
int main()
{ int a[50], n, i, j, temp;
  printf("Enter the no. of elements");
  scanf("%d",&n);
  printf("Enter list of elements to be sorted");
  for(i=0; i<n; i++)
  { scanf("%d",&a[i]); }
  for(i=0; i<n-1; i++) // No. of passes
  {
    for(j=0; j<n-1-i; j++)
    {
      if(a[j]>a[j+1])
      {
        temp=a[j];
        a[j]=a[j+1];
        a[j+1]=temp;
      }
    }
  }
  printf("The sorted array : ");
  for(i=0; i<n; i++)
  {
    printf("%d",a[i]);
  }
  return 0;
}
```

inside one pass.

$n=6$, $j=0$

i	$j<6-1-i$	j
$i=0$	$j<6-1-0 \Rightarrow j<5$	$j=0,1,2,3,4$
$i=1$	$j<6-1-1 \Rightarrow j<4$	$j=0,1,2,3$
$i=2$	$j<6-1-2 \Rightarrow j<3$	$j=0,1,2$
$i=3$	$j<6-1-3 \Rightarrow j<2$	$j=0,1$
$i=4$	$j<6-1-4 \Rightarrow j<1$	$j=0$

Selection Sort

Max no. of passes :- n-1

	i=0 ✓ PASS-1					i=1 PASS-2				i=2 PASS-3			i=3 PASS-4		i=4 PASS-5	
	j=1	j=2	j=3	j=4	j=5	j=2	j=3	j=4	j=5	j=3	j=4	j=5	j=4	j=5	j=5	sorted array
0	40	20	20	20	20	10	10	10	10	10	10	10	10	10	10	10
1	20	40	40	40	40	40	40	40	30	20	20	20	20	20	20	20
2	50	50	50	50	50	50	50	50	50	50	50	40	30	30	30	30
3	60	60	60	60	60	60	60	60	60	60	60	60	60	50	40	40
4	30	30	30	30	30	30	30	30	40	40	40	50	50	60	60	50
5	10	10	10	10	10	20	20	20	20	30	30	30	40	40	50	60
	Ex				Ex			Ex	Ex		Ex	Ex	Ex	Ex	Ex	

/* Program to sort an array using selection sort*/

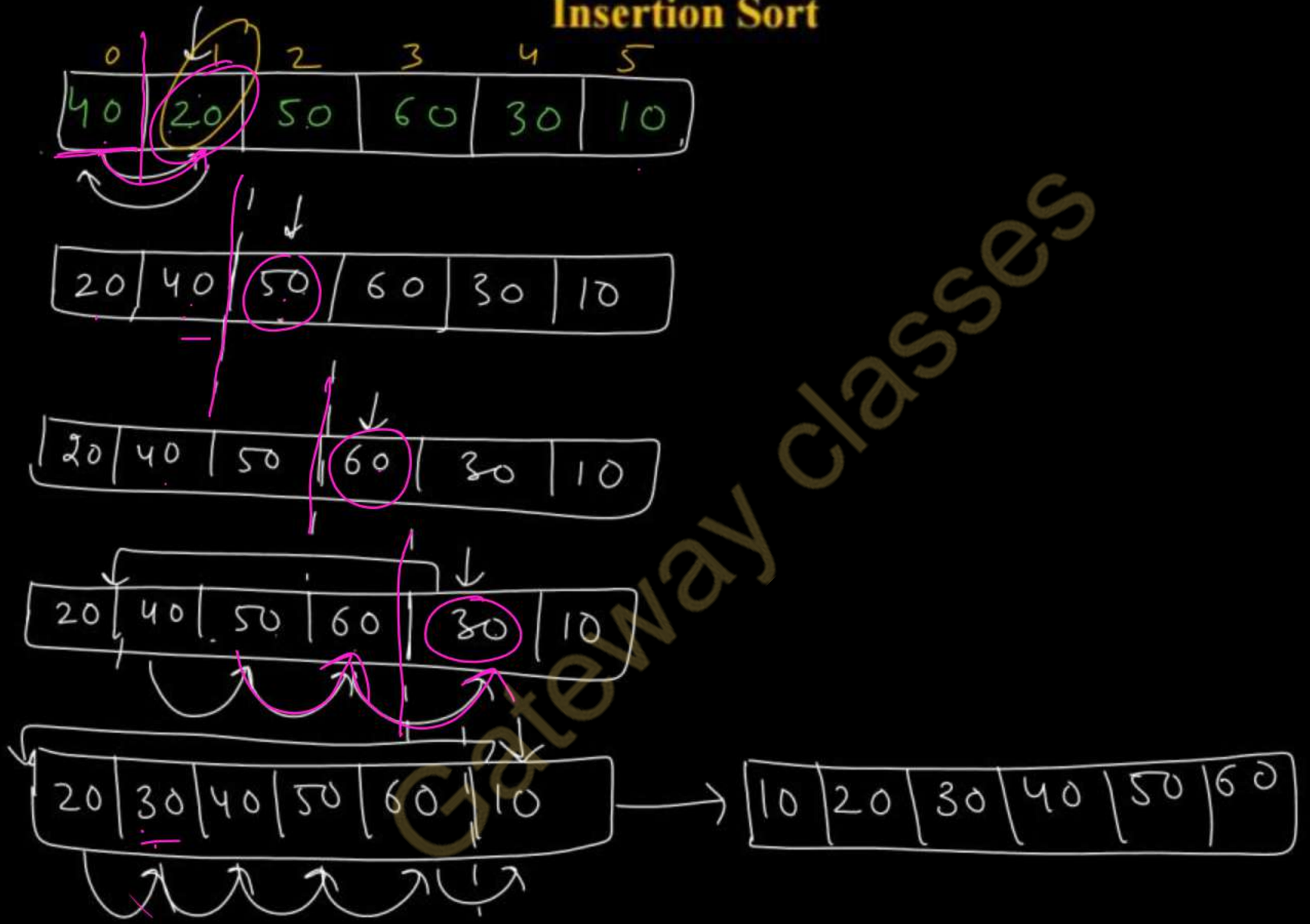
```
#include<stdio.h>
int main()
{ int a[50], n, i, j, temp;
  printf("Enter the no. of elements");
  scanf("%d",&n);
  printf("Enter the list of elements");
  for(i=0; i<n; i++)
  { scanf("%d",&a[i]);}

  for(i=0; i<n-1; i++); // No. of passes
  { for(j=i+1; j<n; j++)
    { if(a[i]>a[j])
      { temp=a[i];
        a[i]=a[j];
        a[j]=temp;
      }
    }
  }

  printf("The sorted array:");
  for(i=0; i<n; i++)
  {
    printf("%d", a[i]);
  }
  return 0;
}
```

Inside one pass.

Insertion Sort



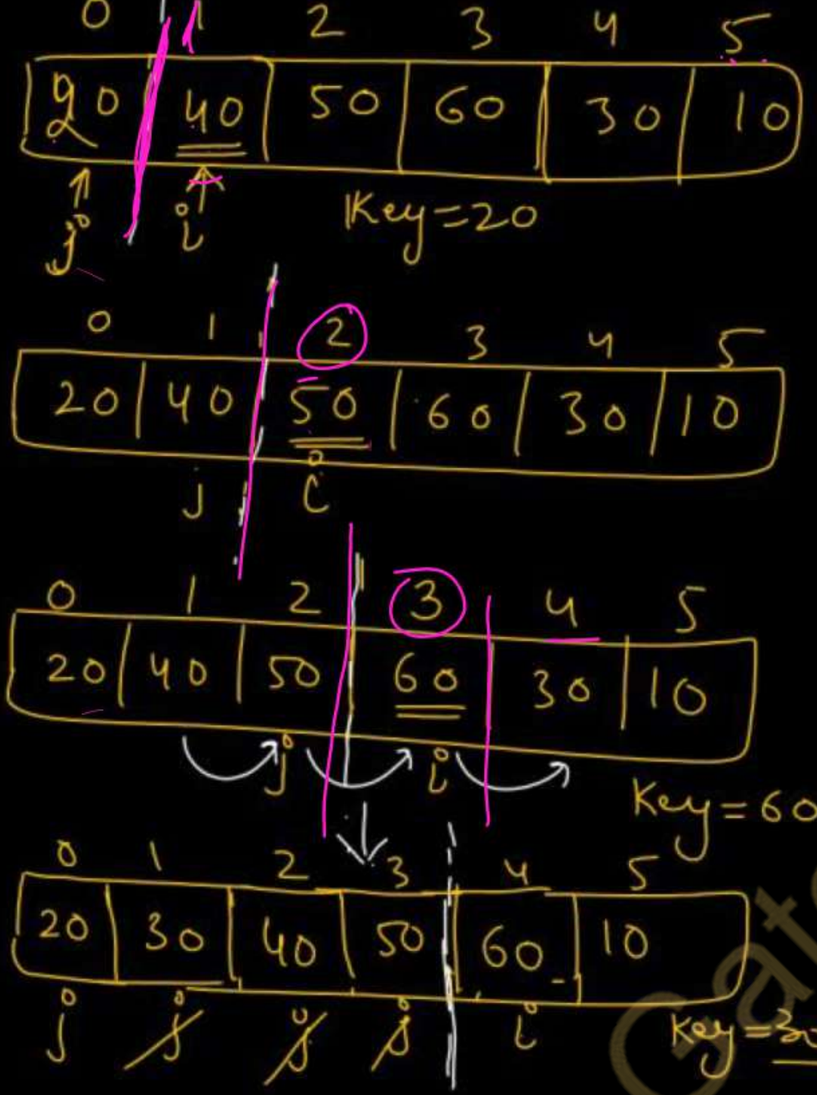
/* Program to sort an array using insertion sort*/

```
#include<stdio.h>
int main()
{ int a[50],n,i,j, Key;
  printf("Enter no. of elements");
  scanf("%d",&n);
 printf("Enter array");
 for(i=0;i<n;i++)
  {
     scanf("%d",&a[i]);
  }
for(i=1; i<n; i++)
{
     Key = a[i];
     j=i-1;
  while(j>=0 && a[j]>Key)
  {
        a[j+1] = a[j];
            j=j-1;
  }
  a[j+1] = Key;
}
printf("The sorted array is:");
 for(i=0;i<n;i++)
  { printf("%d", a[i]);
  }
 return 0;
}
```

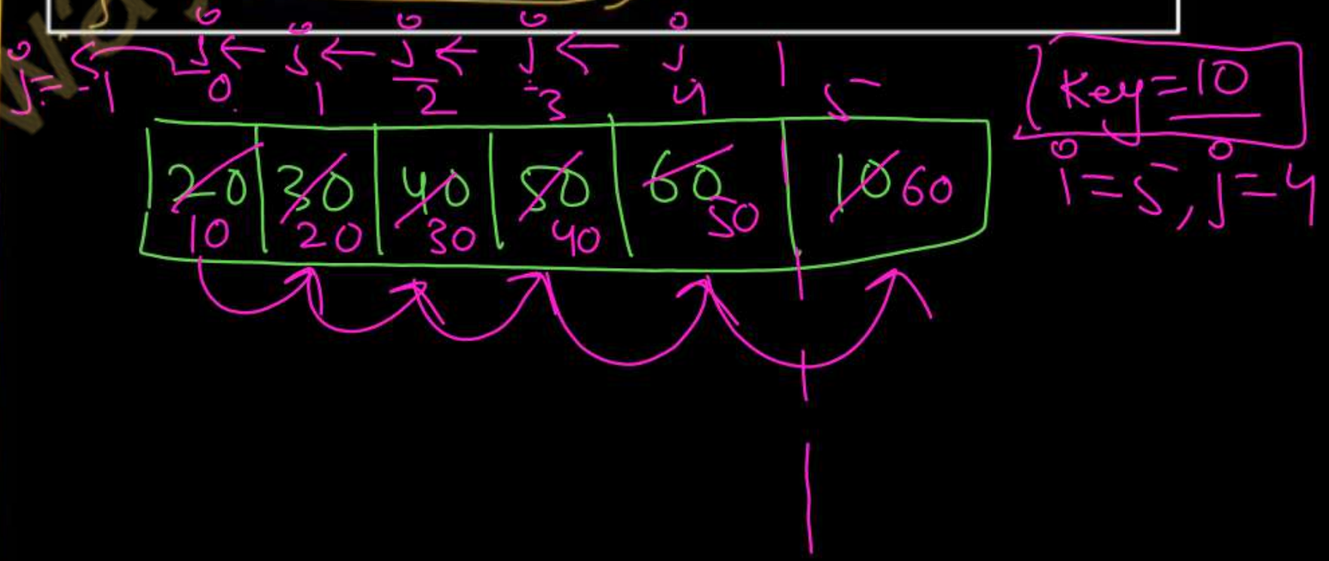
code for insertion sort

/* Program to sort an array using insertion sort*/

n=6




```
for(i=1; i<n; i++)
{
    Key = a[i];
    j = i-1;
    while(j>=0 && a[j]>Key)
    {
        a[j+1] = a[j];
        j = j-1;
    }
    a[j+1] = Key;
}
```



AKTU PYQs

1. Define sorting. Explain the bubble sort technique and write the program to implement it. (AKTU 2023-24, 2018-19)
2. Write the program to implement selection sort. (AKTU 2023-24, 2021-22)
3. Write a program in C to implement bubble sort. (AKTU 2022-23)
4. Why is sorting process required?(AKTU 2022-23)
5. Explain the bubble sort technique with the help of example list {5,7,2,1,3,6}. (AKTU 2023-24)
6. Explain the selection sort technique with the help of example. (AKTU 2020-21)

AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-4: Lecture-5

Topics to be Covered

- Passing arrays to function
- Pointer to structure
- Searching
  - Linear Search
  - Binary Search
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

AKTU PYQs

1. Write a program to print the greatest number in an array using array passing to function concept.(AKTU 2023-24)
2. Write a program to find the presence of an element in an array of n number of elements using binary search method. (AKTU 2023-24)
3. Compare linear search and binary search in terms of complexity. (AKTU 2022-23,2018-19)
4. What is searching? Write a program to search an element using linear search. (AKTU 2022-23)
5. Differentiate linear search and binary search. (AKTU 2018-19)

Array as Parameter passing into function

```
#include<stdio.h>
void display(int a[], int n)
{
    int i;
    for(i=0; i<=n-1; i++)
    {
        printf("%d", a[i]);
    }
}
```

```
int main()
{
    int a[] = {3,8,1,6,4,2};
    display(a, 6);
    return 0;
}
```

a → Name of array (base address of array)

6 → No. of elements

0	1	2	3	4	5
3	8	1	6	4	2
1000	1002	1004	- - -		

Program to find largest number of array using array passing in function

(AKTU 2023-24)

```
#include<stdio.h>

int largest(int a[], int n)
{
  int max = a[0], i;
  for(i=1; i<=n-1; i++)
  {
    if( max<a[i])
        max=a[i];
  }
  return max;
}
```

```
int main()
{
  int a[50], n, result, i;
  printf("Enter no. of elements");
  scanf("%d",&n);
  for(i=0; i<n; i++)
  {
    scanf("%d",&a[i]);
  }
  result = largest(a, n);
  printf("The largest element is=%d",result);
  return 0;
}
```

Creating Pointers to Structure

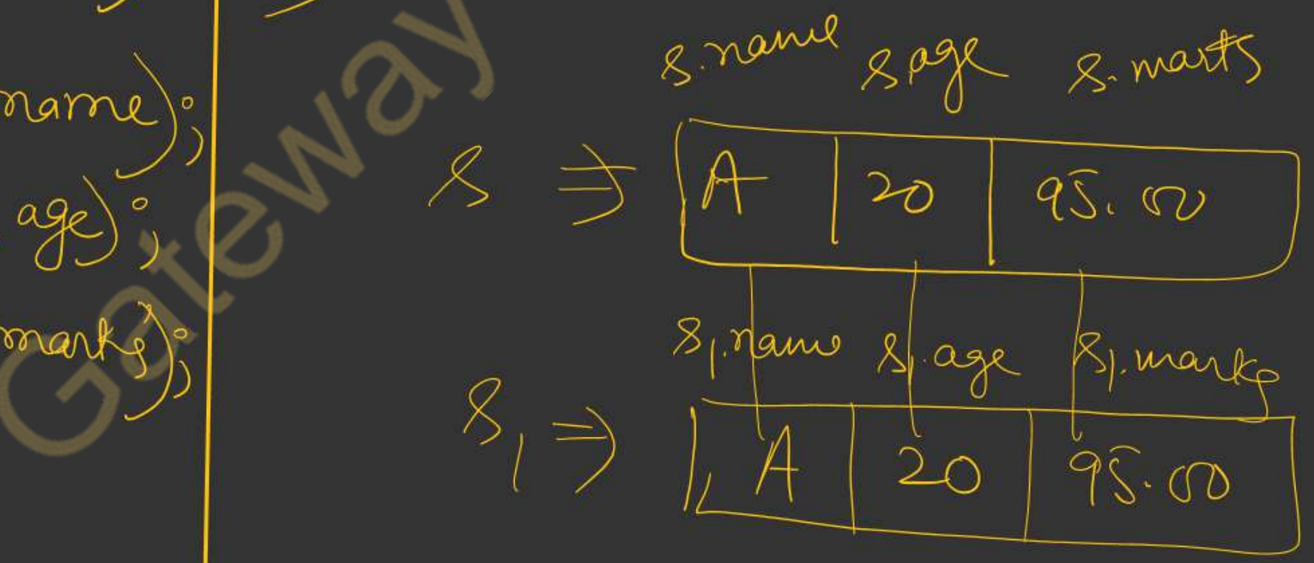
(→) indirection operator

```
#include<string.h>
#include<stdio.h>
struct student
{
  char name[30];
  int age;
  float marks;
};
int main()
{
  struct student s, *p;
  strcpy(s.name,"A");
  s.age=20;
  s.marks=90.50;
  p=&s
```

Pointer to structure

```
  printf("Name=%s", p->name);
  printf("Age=%d", p->age);
  printf("Marks=%f", p->marks);
  return 0;
}
```

```c
#include<stdio.h>
struct student
{
    char name[30];
    int age;
    float marks;
};
void display(struct student s1)
{
    printf("Name=%s",s1.name);
    printf("Age=%d",s1.age);
    printf("Marks=%f",s1.marks);
}
int main()
{
    struct student s={"A",20,95.00};
    display(s);
    return 0;
}
```



Searching

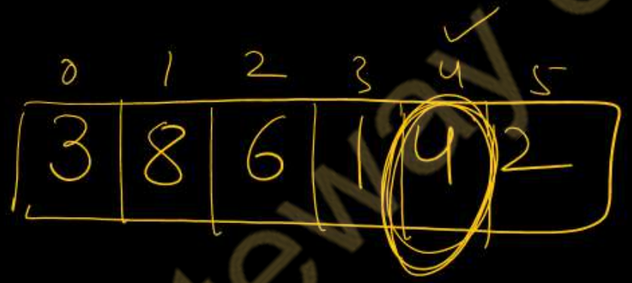
Searching is a process of locating a particular element present in a given set of elements.

It is possible that the search for a particular element is unsuccessful if that element does not exist.

There are two basic techniques available for searching.

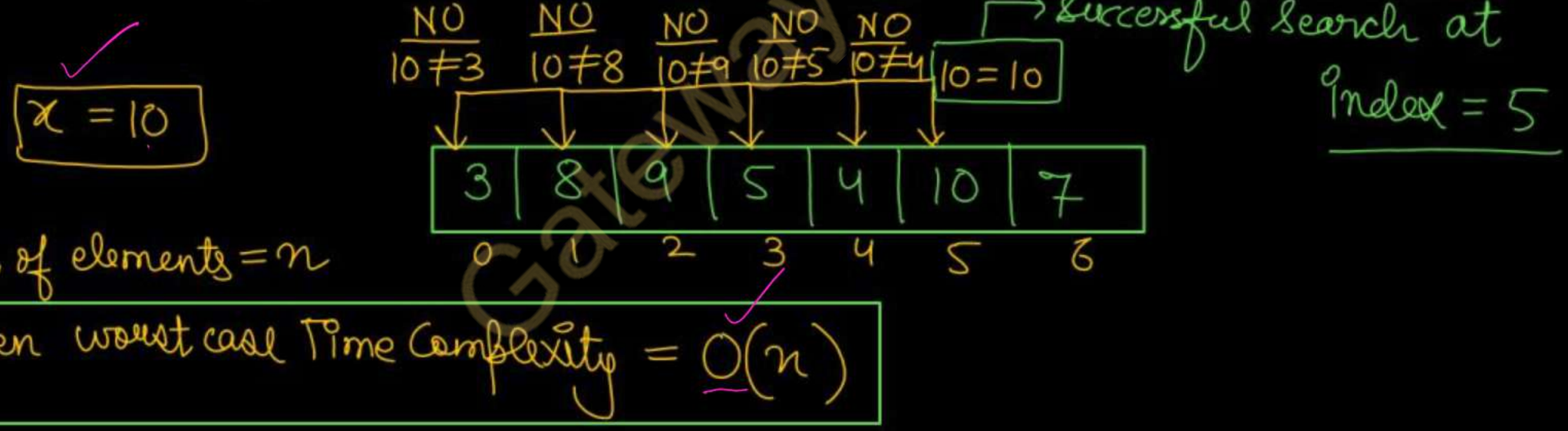
1) Linear Search (Sequential Search) (Basic Search Algo)

2) Binary Search



Linear Search

- In Linear search, we search an element or value in a given array by traversing the array from the starting, till the desired element or value is found.
- It compares the element to be searched with all the elements present in the array.
- when the element is matched successfully, it returns the index of the element in the array, else it return -1.
- Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list.



/* Program to search an element from an array using ~~Binary~~ Search*/

```
#include<stdio.h>
int main()
{
 int a[100], x, n, i, flag=0;
 printf("Enter no. of elements");
 scanf("%d",&n);
 for(i=0;i<n;i++)
 {
  scanf("%d",&a[i]);
 }
printf("Enter element to be searched");
 scanf("%d",&x);
```

Linear

```
for(i=0;i<n;i++)
{
   if(a[i]==x)
   {
      flag=1;
      break;
   }
}
if(flag==1)
    printf("Element found at %dth position",i+1);
else
    printf("Unsuccessful Search");
return 0;
}
```

a

3	8	9	5	4	10	7
0	1	2	3	4	5	6

x = 10

```
for(i=0; i<n; i++)
{
    if(a[i]==x)
    {
        flag=1;
        break;
    }
}
```

i=5

i=0	3≠10
i=1	8≠10
i=2	9≠10
i=3	5≠10
i=4	4≠10
i=5	10=10 True

flag=1

Binary Search

- Binary Search is useful when there are large number of elements in an array and they are sorted. In binary search, to find **key** into the array, we follow the following steps:

- Divide the search space into two halves by finding the middle index "mid"
- Compare the middle element of the search space with the **key**.
- If the **key** is found at middle element, the process is terminated.
- If the **key** is not found at middle element, choose which half will be used as the next search space.
  - If the **key** is smaller than the middle element, then the **left** side is used for next search.
  - If the **key** is larger than the middle element, then the **right** side is used for next search.
- This process is continued until the **key** is found or the total search space is exhausted.

It has a time complexity of O(log n)

$x = 10$

3	8	9	5	4	10	7

↓ sort into ascending order

3	4	5	7	8	9	10
0	1	2	3	4	5	6

$n = 7$

$low = 0, high = 6$

(low) ... (high)

low	high	mid = (low + high)/2
0	6	$(0+6)/2 = 3$; $a[3] \neq 10$ as $(7 \neq 10)$
4	6	$(4+6)/2 = 5$; $a[5] \neq 10$ as $(9 \neq 10)$
6	6	$(6+6)/2 = 6$; $a[6] = 10$ Successful Search.

$low = mid + 1$

$low = mid + 1$

```
if (a[mid] > x)
    high = mid - 1
if (a[mid] < x)
    low = mid + 1
```

/* Program to search an element from an array using Binary Search*/ (AKTU -2023-24)

```
#include<stdio.h>
int main()
{
  int a[100], n, x, mid, low, high;
  int i, flag=0;

  printf("Enter no. of elements");
  scanf("%d",&n);
  printf("Enter array elements");
  for(i=0;i<=n-1;i++)
  { scanf("%d",&a[i]); }
  printf("Enter element to search");
  scanf("%d",&x);

  low=0;
  high=n-1;
  while(low<=high)
  {
    mid=(low+high)/2;
    if(a[mid]==x)
    {
      flag=1;
      break;
    }
    else if(a[mid]>x)
      high=mid-1;
    else
      low=mid+1;
  }
  if(flag==1)
    printf("Element present at %d", mid+1);
  else printf("Unsuccessful search");
  return 0;
}
```

/* Program to search an element from an array using Binary Search *Using Recursion */

```
#include<stdio.h>
int binarysearch(int a[], int, int, int);
int main()
{
int a[100], n, low, high, x, result, i;
printf("Enter no of elements");
scanf("%d",&n);
printf("Input array");
for(i=0;i<n;i++)
{scanf("%d",&a[i]);}
printf("Enter element to search");
scanf("%d",&x);

result= binarysearch(a,0,n-1,x);
if(result==-1)
        printf("Unsuccessful search");
else printf("The element is present at
                %d", result+1);
} return 0;
```

```
int binarysearch(int a[], int low, int high, int x)
{
 if(low<=high)
 {
  mid=(low+high)/2;
  if(a[mid]==x)
        return mid;
  else if(a[mid]>x)
  return binarysearch(a,low,mid-1,x);
  else
  return binarysearch(a,mid+1,high,x);
 }
 else return -1;
}
```

recursive calls to function

AKTU PYQs

1. Write a program to print the greatest number in an array using array passing to function concept.(AKTU 2023-24)
2. Write a program to find the presence of an element in an array of n number of elements using binary search method. (AKTU 2023-24)
3. Compare linear search and binary search in terms of complexity. (AKTU 2022-23,2018-19)
4. What is searching? Write a program to search an element using linear search. (AKTU 2022-23)
5. Differentiate linear search and binary search. (AKTU 2018-19)